



Transformer 最简 Demo 解读

1. 整体架构

Transformer 是一种**序列到序列 (Seq2Seq) 模型**，由 Google 在 2017 年论文 *Attention Is All You Need* 中提出。它完全基于**注意力机制**，摒弃了 RNN/LSTM 的循环结构，训练并行度极高。



2. 核心组件逐行解读

2.1 词嵌入 + 位置编码

```
self.src_embed = nn.Embedding(vocab_size, d_model)
```

- **Embedding** 将离散的 token ID 映射为稠密向量 ($d_model=128$ 维)
- 问题: Self-Attention **不关心位置**——"我-爱-你"和"你-爱-我"对它来说是一样的

- **Positional Encoding** 给每个位置加上独特的"指纹"

```
pe[:, 0::2] = torch.sin(position * div_term) # 偶数位
pe[:, 1::2] = torch.cos(position * div_term) # 奇数位
```

- 不同频率的正弦/余弦波叠加，每个位置产生唯一向量
- 好处：可以外推到训练时未见过的更长序列
- 最终： token嵌入 + 位置编码 → 送入 Encoder/Decoder

2.2 多头自注意力 (Multi-Head Attention)

这是 Transformer 的灵魂，公式为：

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

步骤	含义
QK^T	计算 query 和 key 的 相似度 （点积）
$\div \sqrt{d_k}$	缩放 ，防止点积过大导致 softmax 梯度消失
softmax	将相似度转为 概率分布 （权重）
$\times V$	按权重 加权求和 value

为什么要多头？ 单头只能学到一种"关注模式"，多头让模型同时关注不同位置、不同语义关系（类似 CNN 的多卷积核）。

```

# 拆成多头
Q = w(x).view(B, -1, n_heads, d_k).transpose(1, 2)

# 注意力计算
scores = (Q @ K.transpose(-2, -1)) / math.sqrt(d_k)
attn_weights = F.softmax(scores, dim=-1)
context = attn_weights @ V

# 合并多头
context = context.transpose(1, 2).contiguous().view(B, -1, d_model)

```

2.3 三种注意力使用方式

位置	Q 来源	K/V 来源	遮罩	作用
Encoder Self-Attn	Encoder 自身	Encoder 自身	padding mask	每个词看到所有其他词（双向）
Decoder Self-Attn	Decoder 自身	Decoder 自身	因果遮罩 + padding mask	每个词只能看到 之前 的词（防止作弊）
Decoder Cross-Attn	Decoder	Encoder 输出	padding mask	Decoder 从 Encoder 输出中提取信息

2.4 因果遮罩 (Causal Mask)

```

def generate_causal_mask(sz):
    return torch.tril(torch.ones(sz, sz)) # 下三角矩阵

```

sz=4 时的遮罩矩阵:

<code>[[1, 0, 0, 0],</code>	位置 0 只能看自己
<code>[1, 1, 0, 0],</code>	位置 1 能看 0,1
<code>[1, 1, 1, 0],</code>	位置 2 能看 0,1,2
<code>[1, 1, 1, 1]]</code>	位置 3 能看全部

训练时 Decoder 并行计算所有位置, 但通过遮罩确保位置 i 只能看到 $\leq i$ 的位置, 实现自回归。

2.5 前馈网络 (Feed-Forward)

```
self.net = nn.Sequential(
    nn.Linear(d_model, d_ff),    # 128 → 512 升维
    nn.ReLU(),
    nn.Linear(d_ff, d_model),    # 512 → 128 降维
)
```

- 对每个位置**独立**应用 (Position-wise)
- 升维 → 非线性激活 → 降维: 引入非线性, 增强表示能力

2.6 残差连接 + LayerNorm

每个子层 (Attention / FFN) 后面都是:

```
x = LayerNorm(x + Dropout(Sublayer(x)))
```

- **残差连接 (+x)**: 让梯度直接流过, 解决深层网络的梯度消失
 - **LayerNorm**: 对每个样本的特征维度归一化, 稳定训练
 - 这个模式在代码中出现了 **5 次** (Encoder ×2, Decoder ×3)
-

3. 完整数据流

以 demo 中的**序列复刻任务**为例，输入 `[3, 7, 2]`：

Step 1 – 构造输入

src: [BOS, 3, 7, 2, EOS] → Encoder
tgt_input: [BOS, 3, 7, 2] → Decoder 输入
tgt_out: [3, 7, 2, EOS] → 预测目标

Step 2 – Encoder

Embedding + PosEnc
→ EncoderLayer × 3 (Self-Attn + FFN)
→ encoder_out [1, 5, 128]

Step 3 – Decoder

Embedding + PosEnc
→ DecoderLayer × 3:
① Masked Self-Attn (只看当前及之前)
② Cross-Attn (Q←Decoder, K/V←Encoder)
③ FFN
→ output_proj: [1, 4, 128] → [1, 4, vocab_size]

Step 4 – Loss

CrossEntropy(logits, tgt_out), 忽略 PAD 位置

Step 5 – 推理 (自回归生成)

从 [BOS] 开始，每次生成一个 token，拼回去再跑 Decoder
直到生成 [EOS] 或达到最大长度

4. 运行方式

```
# 安装依赖
pip install torch

# 运行
python transformer_demo.py
```

预期输出：

```
Device: cpu
Generating training dataset...
Epoch 1 | Loss: 2.4558
Epoch 20 | Loss: 1.0737
Epoch 40 | Loss: 0.7253
Epoch 60 | Loss: 0.4481
Epoch 80 | Loss: 0.2893
Epoch 100 | Loss: 0.2054
Epoch 120 | Loss: 0.1770
Epoch 140 | Loss: 0.1326
Epoch 160 | Loss: 0.1108
Epoch 180 | Loss: 0.0890
Epoch 200 | Loss: 0.1009

Input:      [1, 9, 10, 15, 17, 12, 11, 11, 10, 2]
Expected:   [9, 10, 15, 17, 12, 11, 11, 10, 2]
Predicted:  [1, 9, 10, 15, 17, 12, 11, 10, 2]
```

Loss 持续下降，最终模型能完美复刻输入序列。

5. 关键设计决策

决策	原因
<code>d_k = d_model / n_heads</code>	多头总维度不变，计算量与单头相当

决策	原因
缩放因子 <code>√d_k</code>	点积方差随 <code>d_k</code> 增大，softmax 会趋于 one-hot
<code>math.sqrt(d_model)</code> 乘 embedding	防止嵌入值被位置编码淹没
Post-LN vs Pre-LN	原始论文用 Post-LN（残差后归一化）；现代实现多用 Pre-LN（归一化在子层前），本 demo 保持原始风格

6. 从 Demo 到 LLM

实际的大语言模型（如 GPT、LLaMA）在这个基础上做了以下演进：

- **Decoder-Only**：只保留 Decoder 部分，去掉 Encoder 和 Cross-Attention
- **Pre-LayerNorm / RMSNorm**：归一化放在子层前面，更稳定
- **RoPE**：用旋转位置编码代替正弦位置编码
- **GQA / MQA**：分组查询注意力，KV 头数少于 Q 头数，减少显存
- **SwiGLU**：用门控激活函数代替 ReLU
- **大规模训练**：TB 级语料、数千张 GPU、混合精度、梯度累积

但**核心的自注意力机制与本 demo 完全一致**——这也是这个 300 行脚本的价值所在。